



COMP 4300

Intro to Game Programming

Lecture 5
Entity Management
2D Math for Games

Data Management

- One of the most important concepts in all of programming: **separate logic & data**
- Create data structures which **manage** data so your logic code doesn't worry about it
- Complex algorithm shouldn't have to know whether set, vector, map implementation
- Let's make an **Entity Manager**!

Entity Manager

- Uses the **Factory** design pattern
- Entity Manager will handle the creation, storage, and lifetime of **ALL** Entity objects
- EntityManager functions:
 - addEntity, removeEntity, storage, bookkeeping
- Factory should somehow ensure that user **cannot create** its own Entities

Entity Tags

- We may want to **group** Entities by functionality
- Entity is 'tagged' by **string**
 - Tile, Player, Bullet, Enemy
- EntityManager should have functionality for quickly getting Entities **of a given tag**
- Example: loop over all bullets



Entity Implementation

```
class Entity
{
    ComponentTuple    m_components;
    bool              m_alive = true;    // if false, will delete entity
    std::string        m_tag = "default"; // type of entity
    size_t             m_id = 0;         // unique integer id
public:
    Entity() {}

    void          add<T>(args);
    T&             get<T>();
    size_t         id() const;
    bool           isAlive() const;
    void           destroy();
    const std::string& tag() const;
}
```

Component Tuple

```
using ComponentTuple = std::tuple<  
    CTransform,  
    CLifeSpan,  
    CInput,  
    CBoundingBox,  
    CAnimation,  
    CGravity,  
    CState  
>;
```

Entity Manager

- Will **trade** some memory for functionality
- Will store all Entity objects in a **vector**
- Will also store **separate vectors** of Entity objects **by their tag** for quick retrieval
 - `std::map<std::string, EntityVector>`
- We store **double** the number of pointers to save on computation in our engine

Entity Storage Options

1. Store **Entity objects** directly in vector
using `EntityVec = std::vector<Entity>`
2. Store **Smart Pointers** in vector
using `EntityVec = std::vector<std::shared_ptr<Entity>>`
3. Use a **Memory Pool** to store Entity Data
(have a future lecture on this)

`std::vector<Entity>`

- + Avoids overhead of smart pointer
- + Data stored contiguously in memory
- + Simplifies the design overall
- Ownership, references, and lifetime of Entity object are tied to the container
- Removing Entity can be slower (more data)
- Reallocation of owning vector can invalidate existing references to Entity

`std::vector<std::shared_ptr<Entity>>`

- + Shared ownership – can safely refer to Entity in multiple systems **without lifetime worries**
- + **Automatic memory** management
- + Add / Delete from vector is **faster** (less data)
- + Can **modify vector** while still referring to Entities in other parts of the code
- Overhead of smart pointer is **slower**
- Entities **not stored contiguously** in memory
- Slightly more **complex** overall design

Note: can't use `unique_ptr` for even more complex reasons

Entity Storage Options

1. Store **Entity objects** directly in vector
Can have memory issues we want to avoid
2. Store **Smart Pointers** in vector
More robust, slightly slower. **Will use this for now.**
3. Use a **Memory Pool** to store Entity Data
Entity class basically becomes an integer index
This is what we should *actually* do (but later)

Entity Manager

```
1. using EntityVec = std::vector<std::shared_ptr<Entity>>;
2. using EntityMap = std::map    <std::string, EntityVec>;
3. class EntityManager
4. {
5.     EntityVec m_entities;
6.     EntityMap m_entityMap;
7.     size_t    m_totalEntities = 0;
8. public:
9.     EntityManager();
10.    std::shared_ptr<Entity> addEntity(const std::string& tag);
11.    EntityVec& getEntities();
12.    EntityVec& getEntities(const std::string& tag);
13. }
```

Adding Entities

```
1. std::shared_ptr<Entity> EntityManager::addEntity(tag)
2. {
3.     // create a new Entity object
4.     // store it in the vector of all entities
5.     // store it in the map of tag->entityvector
6.     // return the shared pointer pointing to that entity
7. }
```

Adding Entities

```
1. std::shared_ptr<Entity> EntityManager::addEntity(tag)
2. {
3.     auto e = std::make_shared<Entity>(tag, m_totalEntities++);
4.     m_entities.push_back(e);
5.     m_entityMap[tag].push_back(e);
6.     return e;
7. }
```

EntityManager Usage

```
1. EntityManager m_entities;  
  
2. void spawnEnemy()  
3. {  
4.     auto e = m_entities.addEntity("enemy");  
5.     e->add<CTransform>(args);  
6.     e->add<CShape>(args);  
7. }
```

Iterator Invalidation

- Iterator invalidation is **very important**
- Caused by **modifying** the contents of a collection as we are **iterating** through it
- This happens **all the time** in games
 - **Adding**: While iterating through all entities to check for collisions, an explosion happens and we spawn an explosion entity
 - **Removing**: Iterate through entities to check collisions, one dies, so we remove it

C++ Iterator Invalidation

- Vector functions may cause reallocation
 - **Adding** to vector: `push_back`, `insert`, etc
 - Vector may **resize**, recreating internal array
 - Invalidates all pointers and iterators
- Erasing elements may invalidate too
 - **Remove** from vector: `erase`, `pop_back`
 - Invalidates **iterators** and **references** at or after the point of erasing in the internal array

Iterator Invalidation Example

```
1. void sCollision()  
2. {  
3.     EntityVec bullets;  
4.     EntityVec tiles;  
5.     for (auto& b : bullets)  
6.         for (auto& t : tiles)  
7.             if (Physics::IsCollision(b, t))  
8.                 bullets.erase(b)  
9. }
```

// all iterators / values of b after erase may be invalid

Delayed Effects

- One way to avoid iterator invalidation is to **delay the effects** of actions that modify collections until it is safe to do so
- With the EntityManager handling Entity creation and destruction, this becomes easy
- Idea: Only add or remove entities at the beginning of a frame **when it is safe**

Entity Manager

```
1.  typedef std::vector<std::shared_ptr<Entity>> EntityVec;
2.  typedef std::map    <std::string, EntityVec> EntityMap;
3.  class EntityManager
4.  {
5.      EntityVec m_entities;
6.
7.      EntityMap m_entityMap;
8.      size_t    m_totalEntities = 0;
9.  public:
10.     EntityManager();
11.
12.     std::shared_ptr<Entity> addEntity(const std::string& tag);
13.     EntityVec& getEntities();
14.     EntityVec& getEntities(const std::string& tag);
15. }
```

Entity Manager

```
1.  typedef std::vector<std::shared_ptr<Entity>> EntityVec;
2.  typedef std::map    <std::string, EntityVec> EntityMap;
3.  class EntityManager
4.  {
5.      EntityVec m_entities;
6.      EntityVec m_toAdd;
7.      EntityMap m_entityMap;
8.      size_t    m_totalEntities = 0;
9.  public:
10.     EntityManager();
11.     void update();
12.     std::shared_ptr<Entity> addEntity(const std::string& tag);
13.     EntityVec& getEntities();
14.     EntityVec& getEntities(const std::string& tag);
15. }
```

Adding Entities

```
1. std::shared_ptr<Entity> EntityManager::addEntity(tag)
2. {
3.     // give each entity an increasing integer id
4.     auto e = std::make_shared<Entity>(tag, m_totalEntities++);
5.     m_toAdd.push_back(e);
6.     return e;
7. }
```

EntityManager Update

```
// called at beginning of each frame by game engine
// entities added will now be available to use this frame
1. void EntityManager::update()
2. {
3.     for (auto e : m_toAdd)
4.     {
5.         m_entities.push_back(e);
6.         m_entityMap[e->tag()].push_back(e);
7.     }
8.     m_toAdd.clear()
9. }
```

Entity Death / Removal

- With this system, we can easily implement **entity death / removal**
- Entity has **m_alive** bool variable
- Set this to **false** when we want it to die
- EntityManager update() will read this variable and **remove entities if false**
- Prevents removal iterator invalidation

EntityManager Update

```
1. void EntityManager::update()
2. {
3.     for (auto e : m_toAdd)
4.     {
5.         m_entities.push_back(e);
6.         m_entityMap[e->tag()].push_back(e);
7.     }
8.     for (auto e : m_entities)
9.     {
10.        // if e is dead, remove it from m_entities
11.        // if e is dead, remove it from m_entityMap[e->tag()]
12.    }
13.    m_toAdd.clear();
14. }
```

EntityManager Usage

```
1. EntityManager m_entities;

2. void collisions()
3. {
4.     for (auto b : m_entities.getEntities("bullet"))
5.         for (auto e : m_entities.getEntities("enemy"))
6.             if (Physics::CheckCollision(b, e))
7.                 {
8.                     b->destroy(); e->destroy();
9.                 }
10. }
```

Entity Implementation

```
class Entity
{
    ComponentTuple    m_components;
    bool              m_alive = true;    // if false, will delete entity
    std::string        m_tag = "default"; // type of entity
    size_t             m_id = 0;         // unique integer id
public:
    Entity() {}

    void            add<T>(args);
    T&               get<T>();
    size_t          id() const;
    bool            isAlive() const;
    void            destroy();
    const std::string& tag() const;
}
```

Game Main Loop

```
1. void GameEngine::mainLoop()  
2. {  
3.     m_entityManager.update();  
4.     sUserInput();  
5.     sMovement();  
6.     sCollision();  
7.     sRender();  
8.     m_currentFrame++;  
9. }
```

Entity Private Constructor

- We can ensure the safe creation of Entity objects by **only allowing them to be created** by the EntityManager class
- We can make Entity constructor **private** and add EntityManager as friend class
- This way, we **cannot** create Entity objects **outside** of the EntityManager

Entity Private Constructor

- Unfortunately, having a private constructor means that we can no longer use

```
auto e = std::make_shared<Entity>(args);
```

- For private constructor classes, can use

```
auto e = std::shared_ptr<Entity>(new Entity(args));
```

- This is not the preferred way of the creating shared pointer objects, but we make this one exception for our design

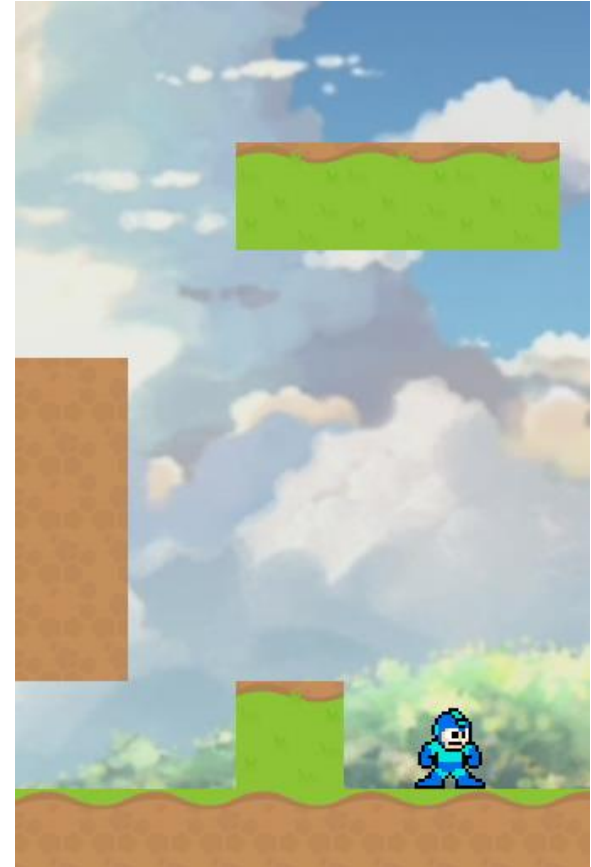
Entity Private Constructor

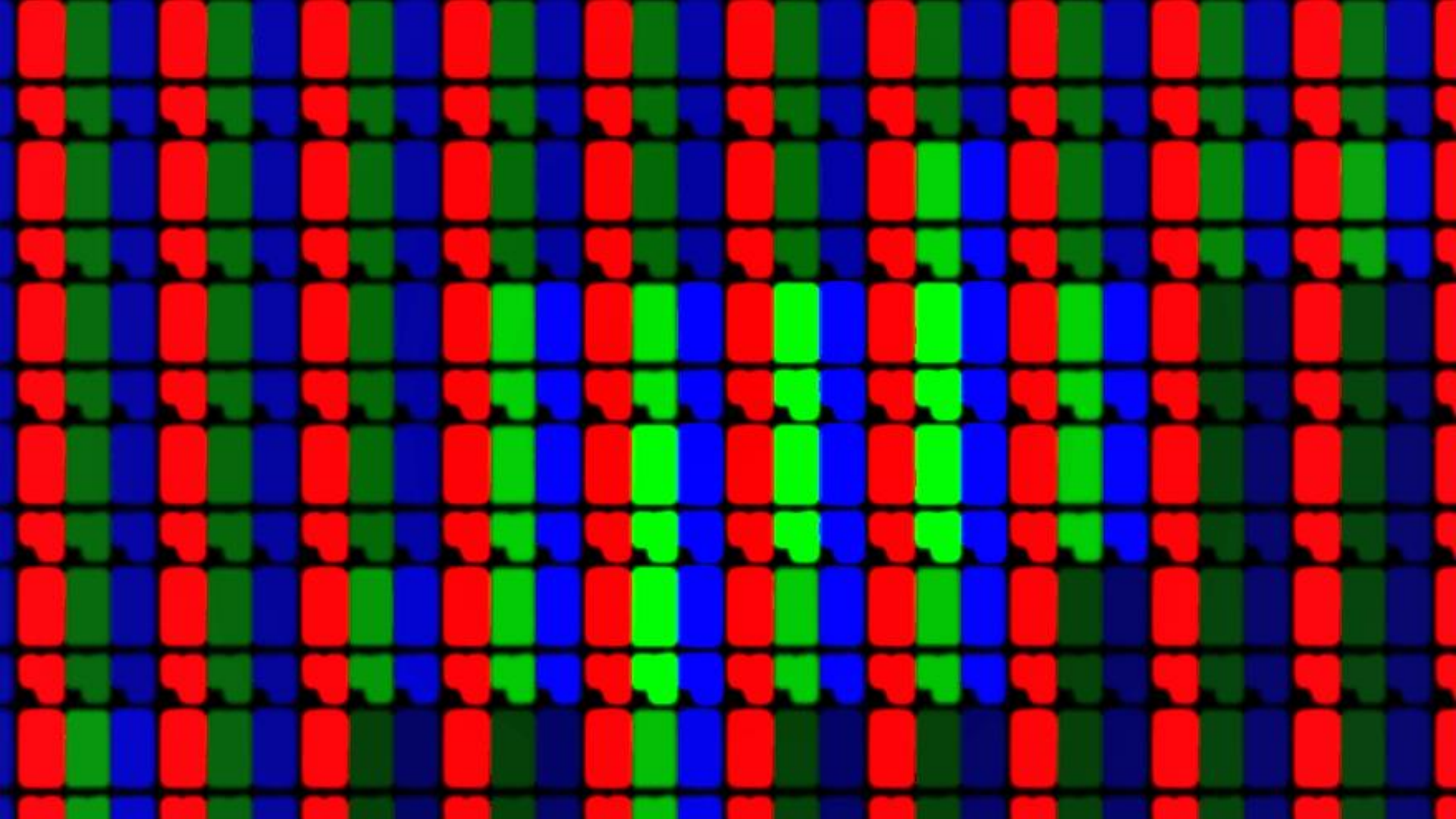
```
1. void GameEngine::sEnemySpawner()  
2. {  
3.     // manually creating entities no longer possible  
4.     // each of these will be a compiler error  
5.     auto e = std::make_shared<Entity>(args);  
6.     auto e = new Entity(args);  
7.     Entity e(args);  
8.  
9.     // must use EntityManager to create entities  
10.    auto e = m_entityManager.addEntity(args);  
11. }
```

2D Game Math

2D Games

- Every game's graphics are drawn to a screen
- Screens are a plane, 2D
- Screens are made of small, discrete elements: pixels
- Pixels are (usually) square
- Pixels have R,G,B elements



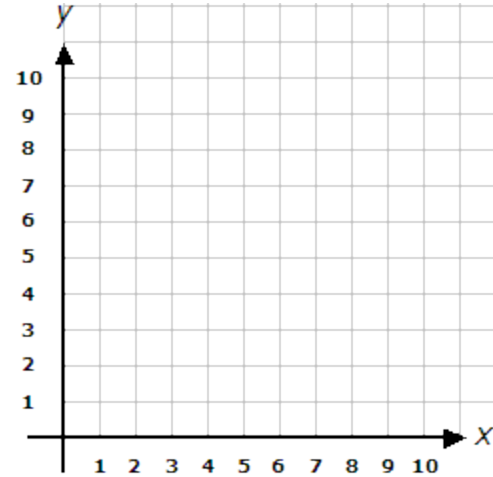


2D Game Colors

- In modern games, we specify the color of an object (pixel, shape, etc) in a specific Color Space of the game / display
- RGB, RGBA, CMYK, Grayscale, Monochrome
- RGBA is most common modern color space
 - Red, Green, Blue, Alpha
- Each pixel gets assigned a color

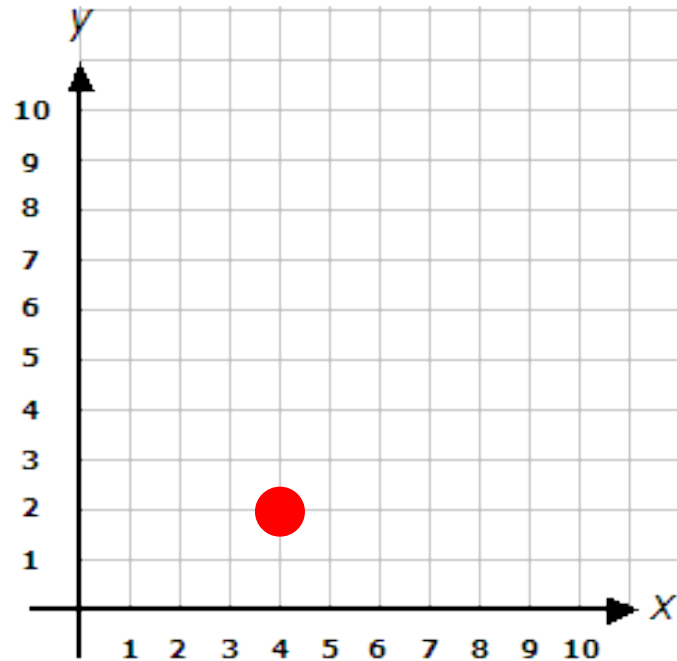
2D Positions

- System display: pixel grid
- Grid is in two dimensions (x,y)
- We can use a Euclidean (x,y) plane to visualize individual positions
- Each integer position is a display pixel
- Note: Displays can be resized / full screen, so pixels can 'appear larger'
- Note: Most APIs have (0,0) in top-left



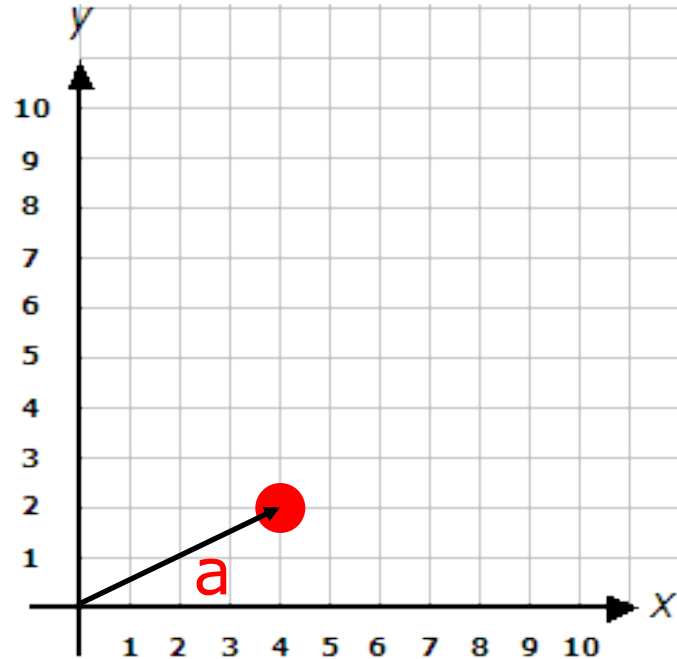
2D Positions

- Example: $a = (4,2)$
- Can represent this 2D value as a vector



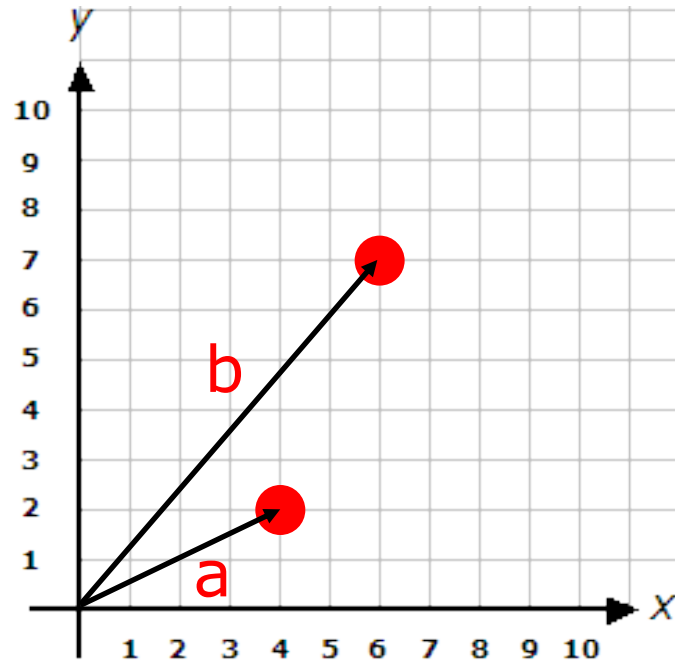
2D Positions

- Example: $a = (4,2)$
- Can represent this 2D value as a vector
- $\text{Vec2} = (X,Y)$
 - Distance from Origin $(0,0)$



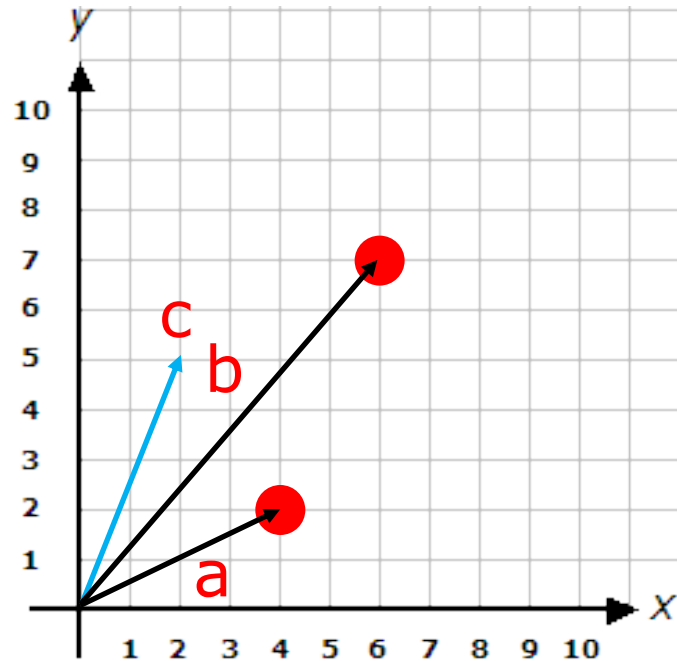
2D Positions

- Another point b: (6,7)



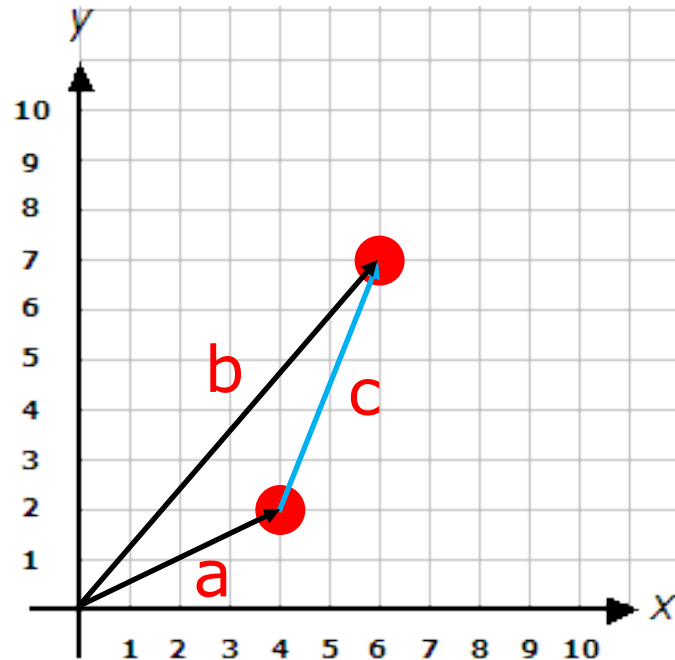
2D Positions

- Another point b: (6,7)
- Let's try some vector math
- Subtraction: $b - a = c$
 - $b(6,7) - a(4,2) = c(2,5)$



2D Positions

- Another point b: (6,7)
- Let's try some vector math
- Subtraction: $b - a = c$
 - $b(6,7) - a(4,2) = c(2,5)$
- Vector Subtraction:
 - Destination - Origin = Distance
- Vector Addition:
 - Origin + Distance = Destination

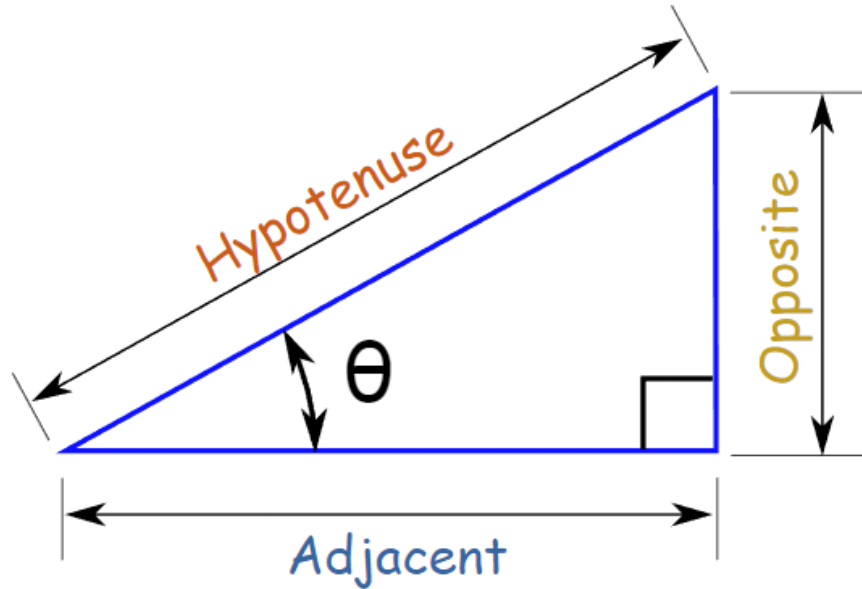


SOH CAH TOA

$$\sin \theta = \frac{\text{Opposite}}{\text{Hypotenuse}}$$

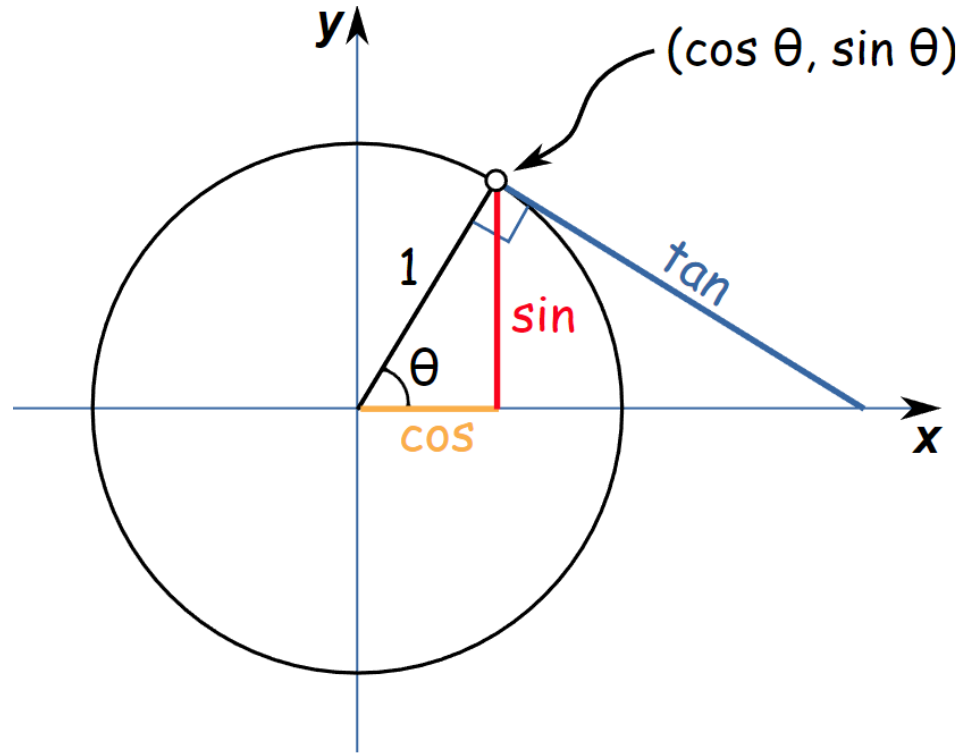
$$\cos \theta = \frac{\text{Adjacent}}{\text{Hypotenuse}}$$

$$\tan \theta = \frac{\text{Opposite}}{\text{Adjacent}}$$

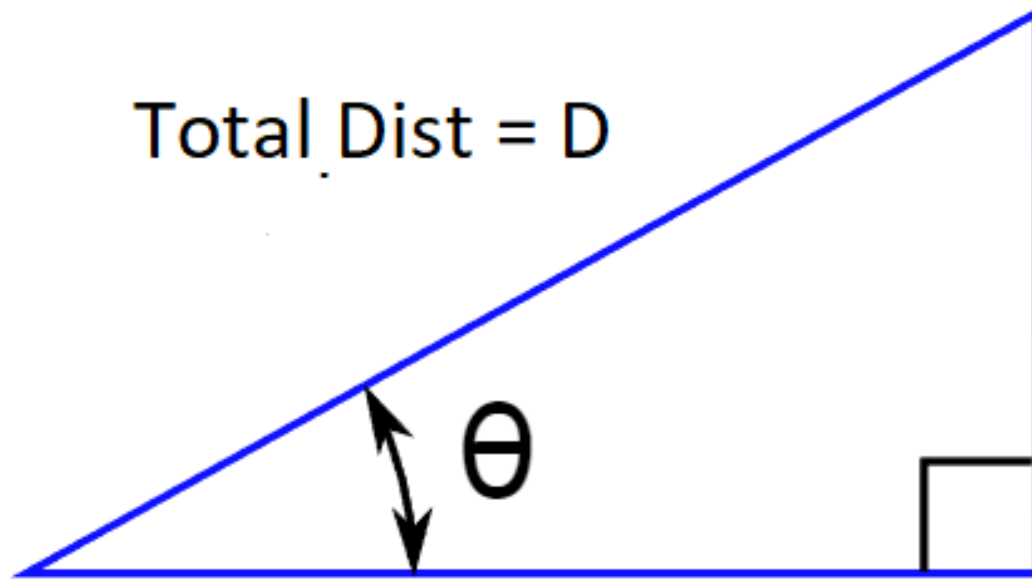


Unit Circle

- Opposite = Y
- Adjacent = X
- Hypotenuse = R
- $\sin(\theta) = Y/R = Y$
- $\cos(\theta) = X/R = X$
- $\tan(\theta) = Y/X$



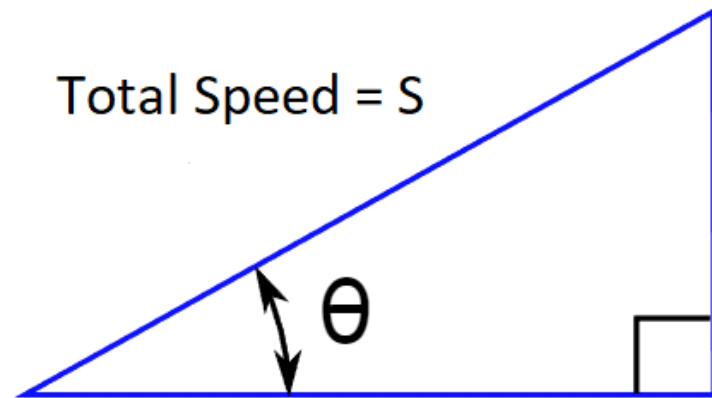
Example: Vector From Angle & Dist



Wanted: (x, y) vector

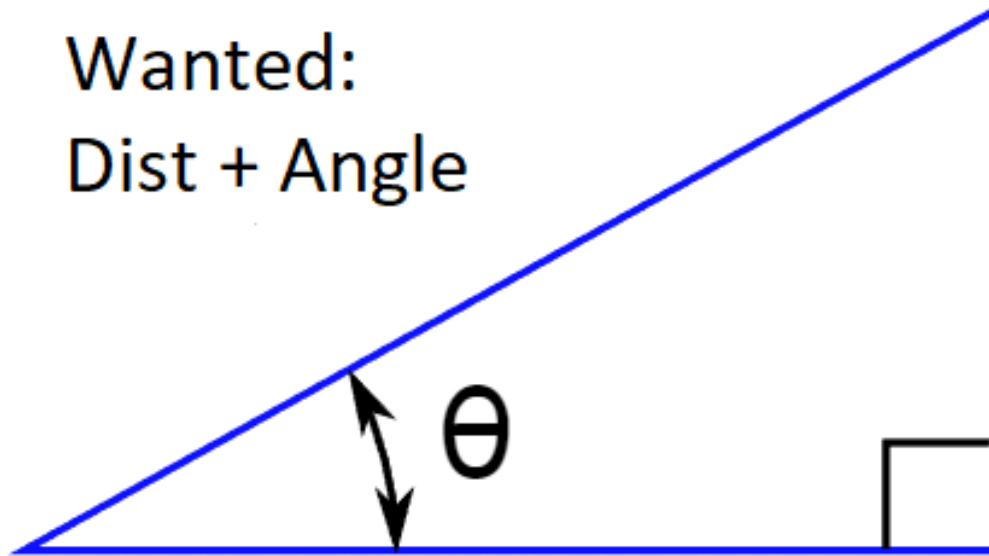
Example: Speed

- Entity speed S units / frame
- Entity to travel at angle of θ
- "Radius" = S
- $\cos(\theta) = X/R$, $\sin(\theta) = Y/R$
 - $\cos(\theta) =$ 'x change if speed was 1'
 - $\sin(\theta) =$ 'y change if speed was 1'
- To get (x,y) velocity vector, must multiply by S
- $\text{Vec2 velocity}(S*\cos(\theta), S*\sin(\theta))$
- Speed is the magnitude (length) of velocity vector



Example: Angle & Dist from Vector

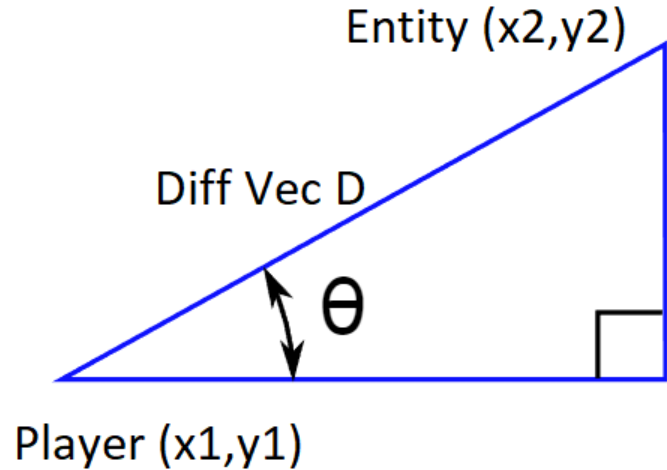
Wanted:
Dist + Angle



Given: (x,y) vector

Example: Dist

- Distance from Player to Entity?
- Difference Vector D
 - $D = \text{Entity} - \text{Player}$
 - $D = (x_2 - x_1, y_2 - y_1)$
 - $\text{Dist} = \text{sqrtf}(D.x * D.x + D.y * D.y) = \text{Pythagorus}$
- Angle from Player to Entity?
 - $\tan(\theta) = Y / X = D.y / D.x$
 - $\theta = \text{atan2f}(d.y, d.x)$ (do not use atan, use atan2)

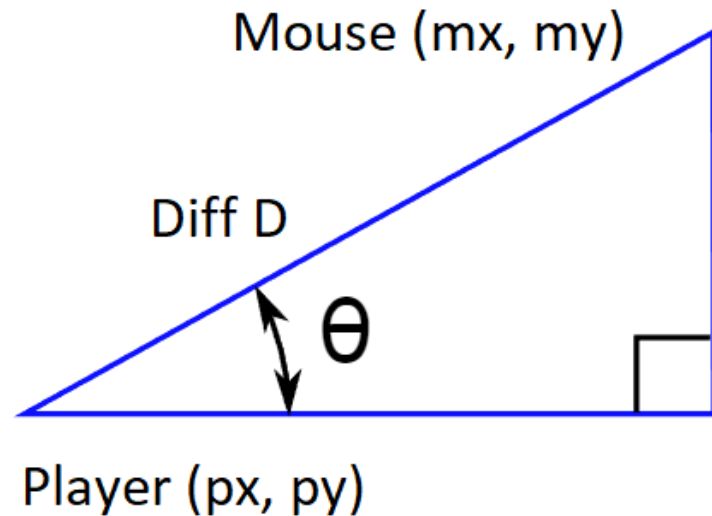


Vector Normalization

- Vector length = distance it travels
 - Distance from Origin = $(0, 0)$
 - $L = \text{sqrtf}(x*x + y*y)$
- Normalizing a vector maintains its angle, but changes its length to 1 (unit vector)
 - $N = \text{Vec2}(V.x / L, V.y / L)$
- Where is this useful?

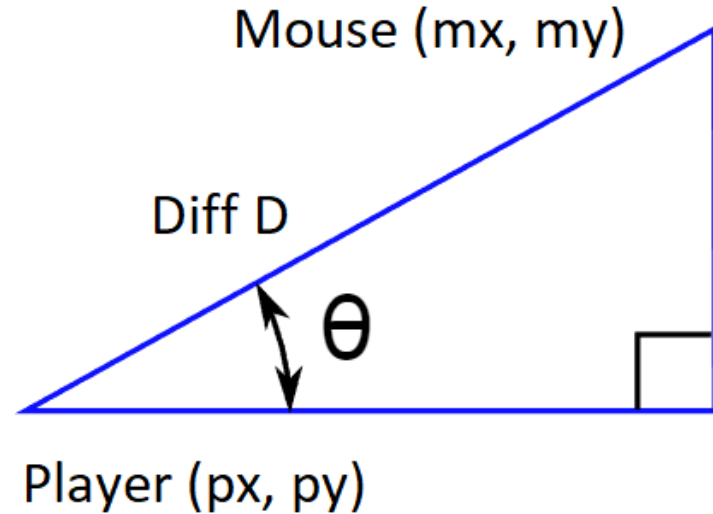
Example: Shooting

- Assignment 2
 - Bullet shoot toward mouse
 - We are given speed scalar S
 - How to get speed vector?
- $D = (mx - px, my - py)$
- D has some length L , we want it to be S
- Normalize D , $N = (D.x / L, D.y / L)$, now has $L=1$
- $\text{Vec2 velocity}(S * N.x, S * N.y)$



Example: Shooting

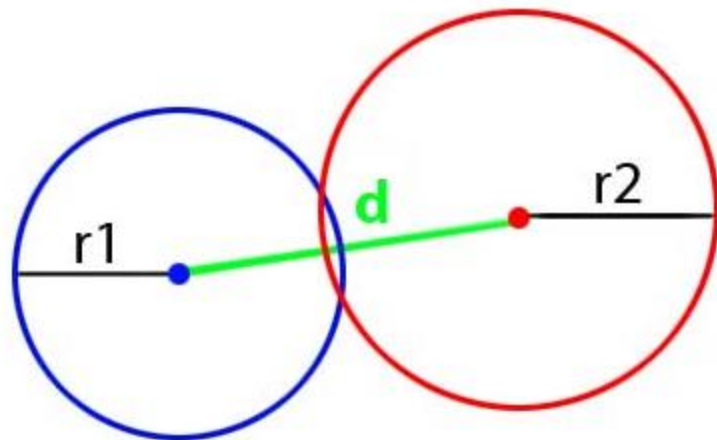
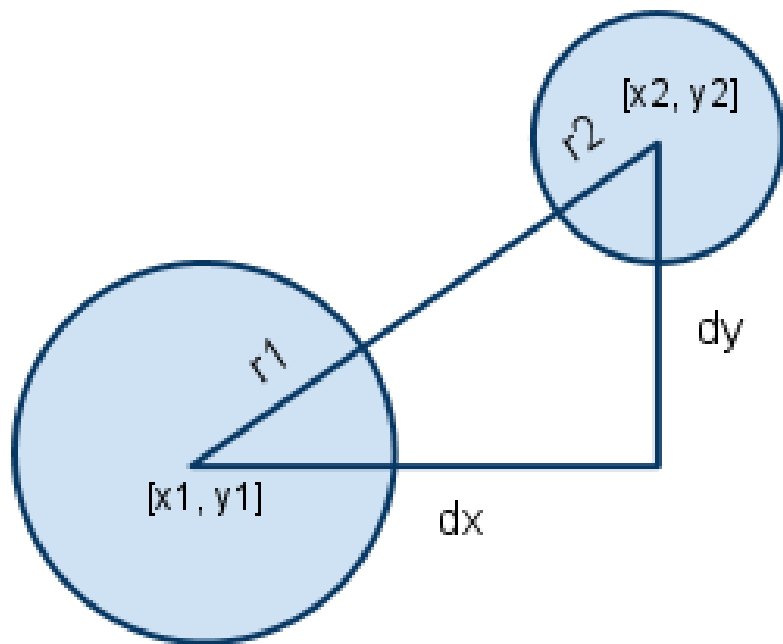
- Assignment 2
 - Bullet shoot toward mouse
 - We are given speed scalar S
 - How to get speed vector?
- $D = (mx - px, my - py)$
- D has angle $\theta = \text{atan2f}(D.y, D.x)$
- Recall: "Vector from Angle & Dist"
- $\text{Vec2 velocity}(S * \cos(\theta), S * \sin(\theta))$



Circle Collisions

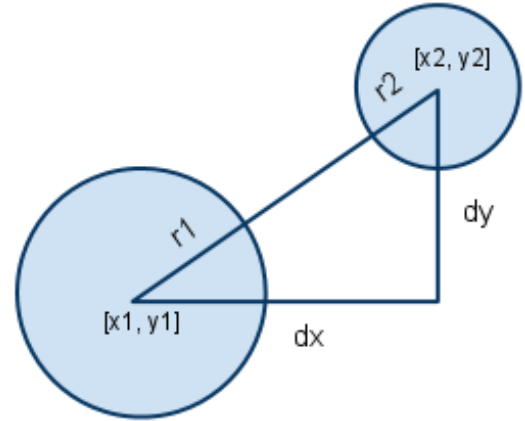
- For Assignment 2, all Entities have been given circular Collision components
- Entity therefore has 2 components which will determine a collision
 - Entity Position (x,y)
 - Entity Collision Radius (r)
- How can we determine a collision?

Circle Collisions



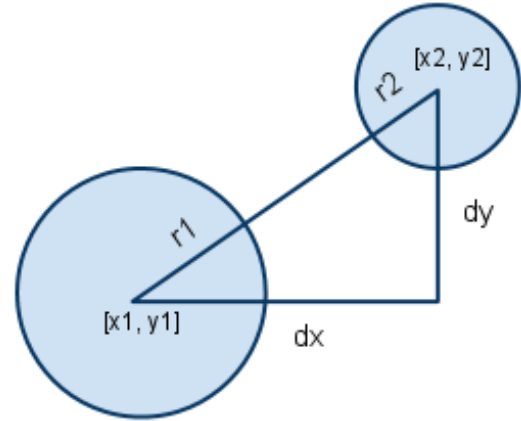
Circle Collisions

- Given two circles
 - $C1(x1,y1,r1)$ $C2(x2,y2,r2)$
- Distance between them
 - Diff Vec $D = (x2-x1,y2-y1)$
 - $Dist = \sqrt{D.x*D.x + D.y+D.y}$
- Collision = $Dist < Radius\ Sum$
 - Collision = $Dist < r1 + r2$



Circle Collisions

- Square Root is expensive
- Needed for actual distance
- Do we need actual distance?
- $\text{sqrt}(D.x*D.x + D.y*D.y) < r1 + r2$
 - We can square both sides
- $\text{DistSq} = D.x*D.x + D.y*D.y$
- $\text{Collision} = \text{DistSq} < (r1+r2)*(r1+r2)$
- Don't calculate actual distance, save the sqrt()



Random Numbers in C++

- Simplest way to generate random numbers in C++ is `rand()` function, which is also present in C
- For serious security, this function is not considered secure / safe, but for this course it is fine for our needs
- Random number generators take a 'seed' which determines the outcome of future RNG
- We can see `rand()` with `srand(int)`
- Default seed is 0, so if not specified, your program will generate the same sequence of RNG every time
- Example: `srand(time(null))` seeds based on current second

Random Number in Range

- Given range $[\text{min}, \text{max}]$ $[\] = \text{inclusive}$
- $\text{diff} = 1 + \text{max} - \text{min};$
- $r = \text{rand}() \% \text{diff}$
 - r is now a random value from $[0, \text{diff}-1]$
- $r = r + \text{min}$
 - add back the minimum to get proper range
- $r = \text{min} + (\text{rand} () \% (1+\text{max}-\text{min}))$

Random In Range Example

- $r = \text{min} + (\text{rand}() \% (1 + \text{max} - \text{min}))$
- Range = [30, 40]
 - $\text{Diff} = 1 + 40 - 30 = 11$
 - $r = \text{rand}() \% 11 = [0, 10]$
 - $r = r + \text{min} = 30 + [0, 10]$
- Range = [20, 20]
 - $r = 20 + (\text{rand}() \% (1 + 20 - 20))$
 - $r = 20 + (\text{rand}() \% 1)$
 - $r = 20$